

COMS20018, Introduction to AI:

Linear Regression and Classification: Part II

Mengyan Zhang

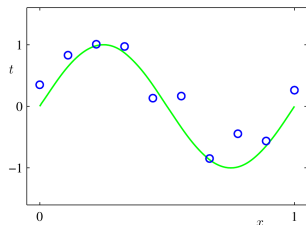
School of Computer Science
University of Bristol

28th September 2026

Agenda

- ▶ Linear regression: recap and extensions
 - ▶ Recap: least squares
 - ▶ High-dimensional inputs
 - ▶ A probabilistic view: maximum likelihood
- ▶ Overfitting
 - ▶ Choosing M : under/overfitting
 - ▶ Regularized least squares
- ▶ Linear classification
 - ▶ Logistic regression & the decision boundary
 - ▶ Fitting with the cross-entropy loss
 - ▶ Evaluation metrics
- ▶ A general view: loss functions
- ▶ Optimisation
 - ▶ Gradient descent & stochastic gradient descent (SGD)

Recap: linear regression & least squares



Bishop, *PRML* (2006), Fig. 1.2: $N=10$ points sampled from $\sin(2\pi x)$ plus random noise.

- **Problem:** given data $\{(x_n, t_n)\}_{n=1}^N$, learn $y(x, \mathbf{w}) = \mathbf{w}^\top \phi(x)$ to predict t for a new x .
- **Least squares:** choose $\mathbf{w}$ to minimise the sum-of-squares loss

$$L_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - y(x_n, \mathbf{w})\}^2$$

- **Closed-form solution** (the normal equations): with design matrix $\Phi_{nj} = \phi_j(x_n)$ and targets $\mathbf{t} = (t_1, \dots, t_N)^\top$,

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \Phi^\dagger \mathbf{t}$$

where $\Phi^\dagger$ is the (Moore-Penrose) **pseudo-inverse** of Φ .

⇒ See L02 for the full derivation and a worked example.

High-dimensional inputs

- ▶ In practice the input is a **vector**, $\mathbf{x} = (x_1, \dots, x_D)^\top \in \mathbb{R}^D$, e.g. a house's size, rooms, age, distance to centre.
- ▶ Linear basis function model (Bishop eq. 3.2), with $\phi_0(\mathbf{x}) = 1$:

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$$

- ▶ Design matrix $\Phi_{nj} = \phi_j(\mathbf{x}_n)$ is still $N \times M$, so $\mathbf{w}_{\text{ML}} = \Phi^\dagger \mathbf{t}$ is *unchanged*.
- ▶ Targets can be vectors too, $\mathbf{t}_n \in \mathbb{R}^K$; in this lecture we keep t **scalar**.

See Bishop, *PRML* (2006), Sections 1.4 and 3.1.

A probabilistic view: maximum likelihood

- ▶ Assume the target is the model prediction plus **Gaussian noise**:

$$t = y(\mathbf{x}, \mathbf{w}) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \beta^{-1})$$

where β is the noise **precision** (inverse variance).

- ▶ Equivalently: $p(t \mid \mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t \mid y(\mathbf{x}, \mathbf{w}), \beta^{-1})$.
- ▶ For independent and identically distributed (i.i.d.) data $\{\mathbf{x}_n, t_n\}_{n=1}^N$, the **likelihood** is the product

$$p(\mathbf{t} \mid \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n \mid y(\mathbf{x}_n, \mathbf{w}), \beta^{-1})$$

- ⇒ Likelihood tells us how well a particular choice of model parameters explains the observed data.
- ⇒ High likelihood: observed t_n are close to what the model predicts.

Maximum likelihood: the log-likelihood

- ▶ Recall the likelihood:

$$p(\mathbf{t} \mid \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n \mid y(\mathbf{x}_n, \mathbf{w}), \beta^{-1})$$

- ▶ Taking $\ln$ turns the product into a sum, the **log-likelihood**:

$$\ln p(\mathbf{t} \mid \mathbf{w}, \beta) = -\beta L_D(\mathbf{w}) + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi)$$

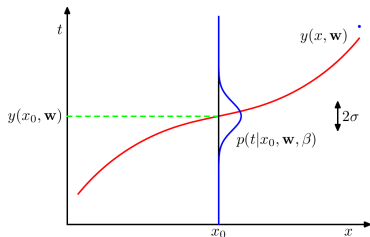
- ⇒ Maximising this over $\mathbf{w}$ (for fixed β) is *exactly the same* as minimising $L_D(\mathbf{w})$: $\mathbf{w}_{\text{ML}}$ is identical to the least-squares solution from last lecture.

Maximum likelihood: what we get for free

- ▶ Least squares only ever gave us a point prediction $y(\mathbf{x}, \mathbf{w})$.
- ▶ The probabilistic view also gives us the **noise level**: setting $\partial \ln p / \partial \beta = 0$,

$$\frac{1}{\beta_{\text{ML}}} = \frac{1}{N} \sum_{n=1}^N \{t_n - y(\mathbf{x}_n, \mathbf{w}_{\text{ML}})\}^2$$

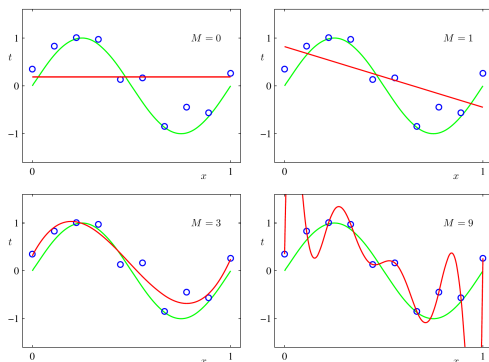
i.e. the residual variance.



⇒ Now we can report a full predictive *distribution* $p(t \mid \mathbf{x}, \mathbf{w}_{\text{ML}}, \beta_{\text{ML}})$, not just a single number, e.g. “ $\pm$ how much” around each prediction.

Bishop, *PRML* (2006), Fig. 1.16: at a new input x_0 , a Gaussian over t centred on $y(x_0, \mathbf{w})$, with $\beta^{-1} = \sigma^2$.

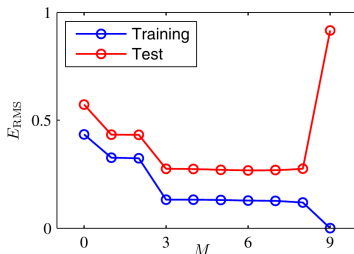
Overfitting vs. underfitting: choosing M



Bishop, *PRML* (2006), Fig. 1.4: least-squares fits (red) of order $M = 0, 1, 3, 9$ to the data from Fig. 1.2 (green = true $\sin(2\pi x)$).

- ▶ $M=0, 1$: too rigid, **underfit**, miss the curve's shape.
- ▶ $M=3$: about right, close to $\sin(2\pi x)$.
- ▶ $M=9$: passes through every data point exactly ($L_D(\mathbf{w}^*) = 0$) but oscillates wildly, **overfits**.

Overfitting vs. underfitting: choosing M



Bishop, *PRML* (2006), Fig. 1.5: root-mean-square error $E_{\text{RMS}} = \sqrt{2L_D(\mathbf{w}^*)/N}$ on the training set vs. a held-out test set, for each M .

- ▶ Training error falls monotonically with M ; more parameters can only fit the training data better (or equally well).
- ▶ Test error is **U-shaped**: too low M underfits, too high M overfits; here $M=3-8$ generalises best.

Why regularize? Overfitting shows up as huge weights

	$M=0$	$M=1$	$M=3$	$M=9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Bishop, *PRML* (2006), Table 1.1: least-squares coefficients for the polynomial fits in Fig. 1.4.

- ▶ As M grows, the magnitude of the fitted coefficients **blows up**.
 - ▶ $M=9$: huge positive and negative weights, finely tuned to cancel exactly at the $N=10$ data points, hence the wild oscillations in between.
- ⇒ Idea: keep the flexible model, but **discourage large weights**.

Regularized least squares

- ▶ Penalise large weights. Add a **regularization term** to the loss:

$$L(\mathbf{w}) = L_D(\mathbf{w}) + \lambda L_W(\mathbf{w}), \quad L_W(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{w}$$

(λ = regularization coefficient). This is **ridge regression**.

- ▶ Still quadratic in $\mathbf{w} \Rightarrow$ still closed-form:

$$\mathbf{w} = (\lambda \mathbf{I} + \Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t}$$

- ▶ $\lambda=0$: back to plain least squares. $\lambda \rightarrow \infty$: $\mathbf{w} \rightarrow 0$, controls model complexity *without* changing M .
- $\Rightarrow$ More general penalty $\sum_j |w_j|^q$: $q=2$ is ridge above; $q=1$ (**lasso**) additionally drives many w_j exactly to zero, automatic feature selection.

Discuss: how can we reduce overfitting?

Discuss with your neighbour (1 min)

Our $M=9$ polynomial overfits. List as many ways as you can to stop a model overfitting.

Discuss: how can we reduce overfitting?

Discuss with your neighbour (1 min)

Our $M=9$ polynomial overfits. List as many ways as you can to stop a model overfitting.

- ▶ **More training data:** for a fixed M , overfitting shrinks as N grows
 - ▶ **Simpler model:** choose M (or fewer features) using a validation set or cross-validation
 - ▶ **Regularization:** penalise large weights, e.g. ridge ($q=2$) or lasso ($q=1$); tune λ on validation data
 - ▶ **For neural networks** (L04 onwards): early stopping, dropout, data augmentation
- ⇒ All trade fit to the training data against how well the model generalises; always judge on held-out data.

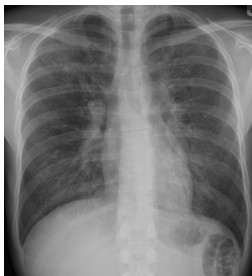
From regression to classification

- ▶ So far: target $t \in \mathbb{R}$ (continuous). Now: assign $\mathbf{x}$ to one of K discrete **classes** $C_1, \dots, C_K$.
- ▶ Two classes ($K=2$): a single scalar $t \in \{0, 1\}$ suffices, with $t=1$ for C_1 and $t=0$ for C_2 .
- ▶ General case, **1-of- K** (one-hot) coding: $\mathbf{t} \in \{0, 1\}^K$ with a single 1 marking the class, e.g. for $K=5$ and class C_2 ,

$$\mathbf{t} = (0, 1, 0, 0, 0)^\top$$

⇒ Today we focus on $K=2$; see Bishop §4.3.4 for the multiclass details.

Classification in practice



Medical diagnosis
pneumonia or healthy?

WIN a FREE prize!!!
Click here now...

spam

Lab 2 moved
Now on Thursday...

not spam

Spam filtering
spam or not spam?

⇒ Same pattern in both: an input $\mathbf{x}$ (an X-ray, an email) $\mapsto$ a **discrete class label** t , here one of two classes.

Image: chest X-ray showing *Pneumocystis* pneumonia. Samir, Wikimedia Commons, public domain.

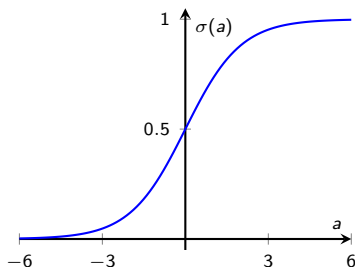
Motivation: from a linear model to a probability

- ▶ First instinct: reuse $y(\mathbf{x}, \mathbf{w}) = \mathbf{w}^\top \phi(\mathbf{x})$ directly? Problem: it's unbounded, so it can't be read off as a probability, and doesn't respect $t \in \{0, 1\}$.
- ⇒ **Logistic regression**: squash the same linear function through a nonlinearity so its output is a valid probability.

- ▶ The **logistic sigmoid**:

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

- ▶ $\sigma(a) \rightarrow 0$ as $a \rightarrow -\infty$,
 $\sigma(a) \rightarrow 1$ as $a \rightarrow +\infty$,
 $\sigma(0) = 0.5$: output always in $(0, 1)$.



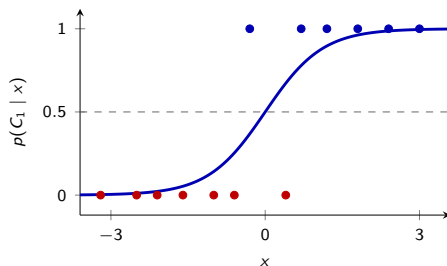
S-shaped: squashes any real a into $(0, 1)$.

Logistic regression

- Model the posterior probability of class C_1 as the **logistic sigmoid** applied to a linear function of the features $\phi(\mathbf{x})$:

$$p(C_1 | \mathbf{x}) = y(\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^\top \phi(\mathbf{x})), \quad \sigma(a) = \frac{1}{1 + e^{-a}}$$

and $p(C_2 | \mathbf{x}) = 1 - y(\mathbf{x}, \mathbf{w})$.



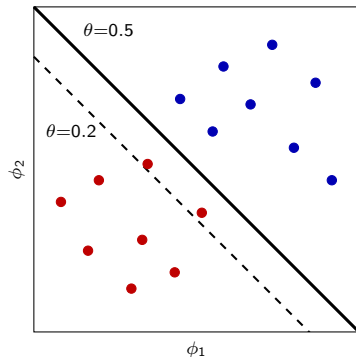
- 1-D example, $\phi(x) = (1, x)^\top$: dots are the labels t_n (C_2 at 0, C_1 at 1)
- The fitted curve $\sigma(w_0 + w_1 x)$ gives a *probability*, not a hard label
- Near the overlap it is uncertain (≈ 0.5)

Decision boundary

- Predict C_1 if $p(C_1 | \mathbf{x}) > \theta$. Since σ is monotonic, the boundary $p(C_1 | \mathbf{x}) = \theta$ is

$$\mathbf{w}^\top \phi(\mathbf{x}) = \ln \frac{\theta}{1 - \theta}$$

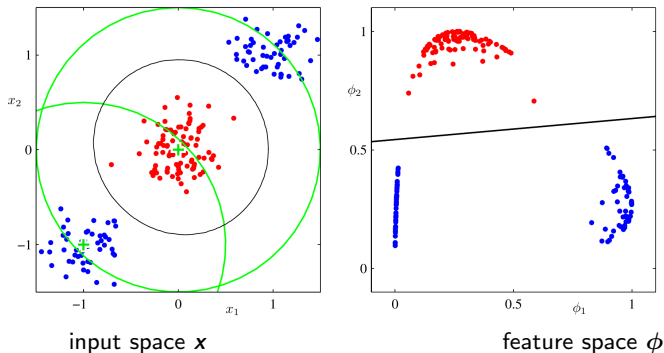
- $\theta = 0.5$ (equal cost for both mistakes, minimises error rate):
 $\mathbf{w}^\top \phi(\mathbf{x}) = 0$
- Other θ just **shift** the boundary parallel, e.g. missing pneumonia is costly $\Rightarrow$ lower θ , flag more patients



Solid: $\theta=0.5$. Dashed: $\theta=0.2$, shifted towards C_2 , so more points are labelled C_1 .

See Bishop, *PRML* (2006), Sections 1.5 and 4.3.2.

Linear in feature space, curved in input space



Bishop, *PRML* (2006), Fig. 4.12.

- ▶ Two Gaussian basis functions $\phi_1(x), \phi_2(x)$ (centres: green crosses) map each x to a point (ϕ_1, ϕ_2) .
- ▶ Right: in ϕ -space a **straight line** separates the classes. Left: the same boundary, drawn in x -space, is the black **circle**.

Fitting logistic regression: the cross-entropy loss

- ▶ With $t_n \in \{0, 1\}$, the likelihood for a single point is **Bernoulli**:

$$p(t_n | \mathbf{x}_n) = \begin{cases} y_n & t_n = 1 \\ 1 - y_n & t_n = 0 \end{cases} = y_n^{t_n} (1 - y_n)^{1-t_n}$$

with $y_n = \sigma(\mathbf{w}^\top \phi_n) = p(C_1 | \mathbf{x}_n)$ and $\phi_n \equiv \phi(\mathbf{x}_n)$: the exponents just switch between the two cases.

- ▶ For the whole (i.i.d.) dataset, the likelihood is the **product**

$$p(\mathbf{t} | \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}$$

- ▶ Taking $-\ln$ turns that product into a sum, the **cross-entropy loss function**:

$$L(\mathbf{w}) = -\ln p(\mathbf{t} | \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

Fitting logistic regression: minimising the loss

- ▶ The gradient of the cross-entropy loss is

$$\nabla L(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n$$

exactly the same form as the least-squares gradient earlier, with y_n now the sigmoid output instead of $\mathbf{w}^\top \phi_n$!

- ⇒ But y_n is nonlinear in $\mathbf{w}$; no closed-form solution this time. Minimise $L(\mathbf{w})$ iteratively: gradient descent or SGD (end of this lecture), or Newton-Raphson (*iteratively reweighted least squares*, Bishop §4.3.3).

Discuss: is 99% accuracy good?

Discuss with your neighbour (1 min)

A spam filter is tested on 1000 emails, of which only 10 are spam. It labels every email “not spam”, so its accuracy is $990/1000 = 99\%$.

Is this a good filter? What is it missing?

Discuss: is 99% accuracy good?

Discuss with your neighbour (1 min)

A spam filter is tested on 1000 emails, of which only 10 are spam. It labels every email “not spam”, so its accuracy is $990/1000 = 99\%$.

Is this a good filter? What is it missing?

- ▶ It never catches a single spam email. Accuracy hides this, because “not spam” dominates the data (**class imbalance**).
- ⇒ We need metrics that count each *kind* of mistake separately: the **confusion matrix**.

How to evaluate a classification task: evaluation metrics

Predicted			
		+	-
Actual	+	True positive (TP)	False negative (FN)
	-	False positive (FP)	True negative (TN)

precision

recall

Positive (+) = the class we care about,
e.g. spam or pneumonia.

- ▶ **Accuracy** = $\frac{TP + TN}{N}$: misleading when classes are imbalanced (previous slide)
- ▶ **Precision** = $\frac{TP}{TP + FP}$: of the points *predicted* positive, how many are actually positive?
- ▶ **Recall** = $\frac{TP}{TP + FN}$: of the *actually* positive points, how many did we catch?
- ▶ **F1** = $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$: their harmonic mean, high only if both are

⇒ As with regression, always evaluate on held-out (validation or test) data, not on the training set used to fit $\mathbf{w}$.

A general view: loss functions

- ▶ Both regression and classification fit $\mathbf{w}$ the same way: choose a **loss function** $L(\mathbf{w})$ that measures how wrong the model's predictions are, then minimise it.
- ▶ **Regression**: sum-of-squares loss

$$L_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - y(\mathbf{x}_n, \mathbf{w})\}^2, \quad y_n = \mathbf{w}^\top \phi_n$$

- ▶ **Classification**: cross-entropy loss

$$L_D(\mathbf{w}) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}, \quad y_n = \sigma(\mathbf{w}^\top \phi_n)$$

- ▶ Either can add a regularizer: $L(\mathbf{w}) = L_D(\mathbf{w}) + \lambda L_W(\mathbf{w})$ (L_D = fit to the data).
- ⇒ A general pattern, not just these two: many other losses exist, e.g. absolute error (robust to outliers) or hinge loss (support vector machines). The right choice depends on the task; see the extension reading below.

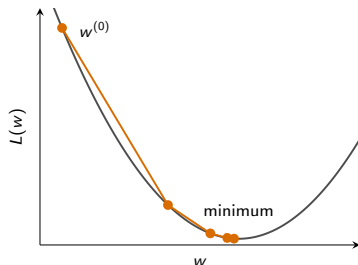
Extension reading, how to choose a loss for a given task: Elharrouss et al., *Loss Functions in Deep Learning: A Comprehensive Review*, arXiv:2504.04242 (2025).

arxiv.org/html/2504.04242v1

Gradient descent

- ▶ The normal equations invert the $M \times M$ matrix $\Phi^\top \Phi$: $O(M^3)$, costly for many features
- ▶ Many losses (e.g. logistic regression's cross-entropy) have *no* closed form
- ▶ Instead, walk **downhill**: repeatedly step against the gradient,

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla L(\mathbf{w}^{(\tau)})$$

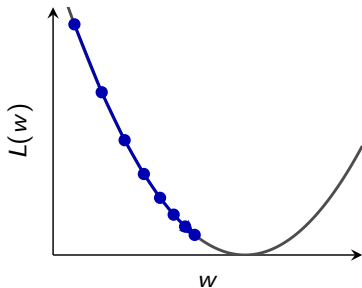


Each step moves downhill; steps shrink as the slope flattens near the minimum.

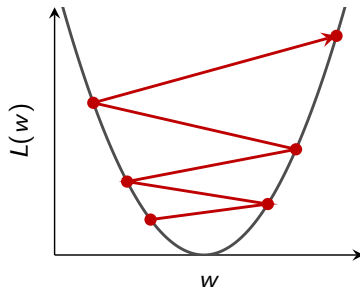
Gradient descent: learning rate and cost

- η = **learning rate**: too small $\Rightarrow$ slow; too large $\Rightarrow$ overshoots and can diverge.

small η : slow



too large η : diverges



$\Rightarrow$ **Batch** GD, $\nabla L(\mathbf{w}) = \sum_{n=1}^N \nabla L_n(\mathbf{w})$, still touches all N points at every step: costly for large $N \Rightarrow$ SGD (next).

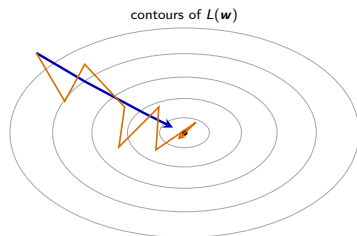
Stochastic gradient descent (SGD)

- **Stochastic** (online): use the gradient of a *single* point's loss $L_n(\mathbf{w})$ per step:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla L_n(\mathbf{w}^{(\tau)})$$

- Least squares, $L_n = \frac{1}{2}(t_n - \mathbf{w}^\top \phi_n)^2$: $\nabla L_n(\mathbf{w}) = (\mathbf{w}^\top \phi_n - t_n) \phi_n$, the **least-mean-squares (LMS)** algorithm.
- Logistic regression: the same update, with $y_n = \sigma(\mathbf{w}^\top \phi_n)$ in place of $\mathbf{w}^\top \phi_n$.

⇒ Scales to huge/streaming datasets (no $\Phi^\top \Phi$, no full-batch gradient); the same idea trains neural networks (L04).



batch GD: smooth

SGD: noisy, but cheap steps

See Bishop, *PRML* (2006), Section 3.1.3.

SGD: the algorithm

```
Input: learning rate  $\eta$ , initial weights  $\mathbf{w}$   
repeat  
| shuffle the  $N$  training points  
| for  $n = 1, \dots, N$  do  
| |  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L_n(\mathbf{w})$   
| end  
until validation loss stops improving, or max epochs  
      reached
```

- ▶ One pass through all N points is one **epoch**.
- ▶ **Shuffle** each epoch, so the updates don't follow the same fixed order every time.
- ▶ **When to stop:** after a fixed number of epochs, or when the loss on a held-out *validation* set stops improving.

Summary

- ▶ **Least squares** = maximum likelihood under Gaussian noise
- ▶ **Regularization** $\lambda L_W(\mathbf{w})$ curbs overfitting
- ▶ **Logistic regression**: $\sigma(\mathbf{w}^\top \phi(\mathbf{x}))$, cross-entropy loss
- ▶ **Evaluation**: confusion matrix, precision, recall, F1
- ▶ **(S)GD**: minimise any loss iteratively, scales to large N

Reading

- ▶ Bishop, C. M., *Pattern Recognition and Machine Learning* (2006). [Free online](#).
 - ▶ Chapter 1: §1.1 (overfitting), §1.2.5 (probabilistic view), §1.5 (decision thresholds)
 - ▶ Chapter 3: §3.1 (maximum likelihood, sequential learning, regularization, multiple outputs)
 - ▶ Chapter 4: §4.3.1–4.3.2 (fixed basis functions, logistic regression)